



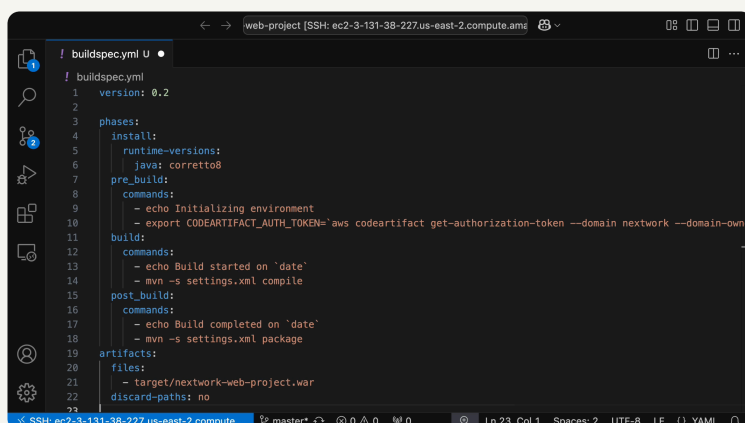
nextwork.org

Continuous Integration with CodeBuild



Muhammad Sadique

<https://www.linkedin.com/in/muhammad-sadique->



```
! buildspec.yml U
! buildspec.yml
1 version: 0.2
2
3 phases:
4   install:
5     runtime-versions:
6       java: corretto8
7   pre_build:
8     commands:
9       - echo Initializing environment
10      - export CODEARTIFACT_AUTH_TOKEN=$(aws codeartifact get-authorization-token --domain nextwork --domain-owne
11  build:
12    commands:
13      - echo Build started on `date`
14      - mvn -s settings.xml compile
15  post_build:
16    commands:
17      - echo Build completed on `date`
18      - mvn -s settings.xml package
19  artifacts:
20    files:
21      - target/nextwork-web-project.war
22  discard-paths: no
23
```



Introducing Today's Project!

I will demonstrate working with AWS CodeBuild which is a CI tool - I helps faster software code delivery by automating the build process for my web app. I am doing this project to learn CI/CD pipeline concept which is paramount for DevOps.

Key tools and concepts

Services I used were: 1. AWS CodeBuild 2. AWS CodeConnections Key concept that i learn was the AWS CodeBuild is a CI tool which is used that automates the software build process. Engineers love continuous integration tools like CodeBuild because you don't have to manually set up and manage any build servers yourself, and you only pay for the compute time you use for building your projects (instead of entire servers that are idle most of the time)

Project reflection

This project took me approximately 8 hours - including multiple breaks. The most challenging part was to complete the project till the end. It was most rewarding to complete the secret mission as well.

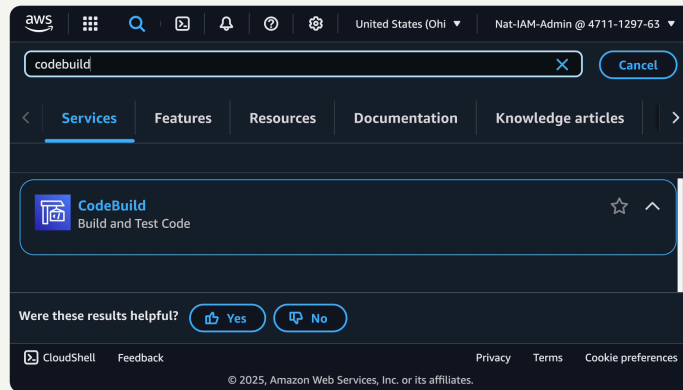
This project is part four of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project tomorrow inshALLAH.



Setting up a CodeBuild Project

AWS CodeBuild is a fully managed continuous integration service that automates the software build process. Think of it as a robot that takes your code, prepares it, tests it, and packages it up, ready for deployment, without you having to do it manually each time. Engineers love continuous integration tools like CodeBuild because you don't have to manually set up and manage any build servers yourself, and you only pay for the compute time you use for building your projects (instead of entire servers that are idle most of the time).

My CodeBuild project's source configuration means that, CodeBuild doesn't know where your web app's code lives! Source means the location of the code that CodeBuild will fetch, compile, and package into a file you can deploy. We're choosing GitHub as our source provider because that's where we've stored our web app's code. By connecting CodeBuild directly to GitHub, we're creating a seamless pipeline where code changes automatically flow into our build process.





Connecting CodeBuild with GitHub

When connecting to GitHub, CodeConnections gives you a few different options, each with their own trade-offs: GitHub App: This is generally the simplest and most secure option. AWS manages the application and connection, reducing the need for you to handle tokens or keys directly. It's recommended for most use cases due to its ease of use and enhanced security.

AWS CodeConnections is like a secure bridge between AWS and your external code repositories. Instead of dealing with the headache of managing API keys, tokens (like GitHub's Personal Access Tokens!), or SSH credentials, CodeConnections handles all that authentication complexity for you - so you can focus on building your application.

Source 1 - Primary

Source provider
GitHub

Credential
✔ Your account is successfully connected by using an AWS managed GitHub App. [Manage](#)

☐ Use override credentials for this project only

Repository
☒ Repository in my GitHub account ☐ Public repository ☐ Git



CodeBuild Configurations

Environment

The environment in AWS CodeBuild refers to the configuration settings that determine how and where your code is built. It's like setting up a virtual workspace with all the necessary tools and resources for your build process to run smoothly. A CodeBuild environment encompasses several key aspects: Operating System: Specifies the OS for the build environment (e.g., Amazon Linux, Ubuntu, Windows Server). Runtime: Determines the programming language and its version (e.g., Java, Python, Node.js). Compute Resources: Defines the computing power allocated to the build, influencing build speed and cost. Buildspec File: A YAML file that outlines the build commands and settings.

Artifacts

Build artifacts are the tangible outputs of our build process. They're what we'll actually deploy to our servers or distribute to users. Our compiled applications, libraries, or any output files from your build need a safe, accessible home after the build finishes. S3 is perfect for this - it's a highly reliable and scalable storage solution that's also in our AWS environment (which makes the artifact easily accessible for deployment later).

Packaging

When setting up CodeBuild, I also chose to package artifacts in a zip file because of smaller size, better management of one tidy package as oppose to multiple files. Zip file keeps the simplicity.

Monitoring

For monitoring, I enabled CloudWatch Logs, which is a monitoring service that collects and tracks logs from AWS services. In this project, CloudWatch will record everything that happens during the build process, including the commands that are run, the



Muhammad Sadique

NextWork Student

nextwork.org

This is incredibly useful for debugging and understanding what went wrong if a build fails.

buildspec.yml

My first build failed because there was no buildspec.yml file in my GitHub repository. The file is needed because it is like a step-by-step instruction manual. It goes through each phase in order - install, pre_build, build, then post_build - running the commands you've specified in each section.

The phases in my buildspec.yml file are: 1. Install - The "prep work" phase - here, we're telling CodeBuild to use Java 8. 2. pre_build - Tasks to do before the main building starts. Here, we're grabbing a security token so we can access our dependencies. 3. Build - Where the actual building happens. We're using Maven (a popular Java build tool) to compile our code. 4. Post_build - The finishing touches after the main build is done. Here, we're packaging everything into a WAR file (a format for web applications).

```
buildspec.yml
1 version: 0.2
2
3 phases:
4   install:
5     runtime-versions:
6       java: corretto8
7   pre_build:
8     commands:
9       - echo Initializing environment
10      - export CODEARTIFACT_AUTH_TOKEN=$(aws codeartifact get-authorization-token --domain nextwork --domain-owner
11  build:
12    commands:
13      - echo Build started on $(date)
14      - mvn -s settings.xml compile
15  post_build:
16    commands:
17      - echo Build completed on $(date)
18      - mvn -s settings.xml package
19  artifacts:
20    files:
21      - target/nextwork-web-project.war
22  discard-paths: no
```



Success!

My second build also failed, but with a different error that said "COMMAND_EXECUTION_ERROR: Error while executing command: mvn -s settings.xml compile. Reason: exit status 1" CodeBuild still can't access the settings.xml file. This usually happens because our CodeBuild service role doesn't have permission to access CodeArtifact, which is needed to download project dependencies. To fix this, we need to grant CodeBuild's IAM role the permission to access CodeArtifact.

To resolve the second error, I attached "codeartifact-nextwork-consumer-policy" to role "codebuild-nextwork-devops-cicd-service-role" After this step the CodeBuild had then sufficient permissions on setting.xml file. The Build succeeded this time.

To verify the build, I checked the S3 bucket that i created for this project and there it was.... The artifact that was created after successful build.



Build started

You have successfully started the following build: nextwork-devops-cicd:39141efa-2201-4fc9-9bcd-33c4d6cf1df1

Developer Tools > CodeBuild > Build projects > nextwork-devops-cicd > nextwork-devops-cicd:39141efa-2201-4fc9-9b...

nextwork-devops-cicd:39141efa-2201-4fc9-9bcd-33c4d6cf1df1

Stop build

Debug build

Retry build

Build status

Status	Initiator	Build ARN
Succeeded	saddi2u@gmail.com	arn:aws:codebuild:us-east-1:619071326582:build/nextwork-devops-cicd:39141efa-2201-4fc9-9bcd-33c4d6cf1df1



Automating Testing

In a project extension, I'm writing a simple unit test for your Java web application.

To add the test script to the build process, I added command in build phase with markers to show when the units test starts or finishes - weather the build phases succeeds or not.

After pushing my code to GitHub, I ran the build - i could see the unit test running and succeeding.

```
$ run-tests.sh M
$ run-tests.sh
1  #!/bin/bash
2
3  echo "==== RUNNING SIMPLE TESTS ====="
4  echo "Test 1: Checking project structure..."
5  if [ -d "src" ]; then
6      echo "✅ PASS: src directory exists"
7  else
8      echo "❌ FAIL: src directory not found"
9      exit 1
10 fi
11
12 echo "Test 2: Checking for web app files..."
13 if [ -f "src/main/webapp/index.jsp" ]; then
14     echo "✅ PASS: index.jsp exists"
15 else
16     echo "❌ FAIL: index.jsp not found"
17     exit 1
18 fi
19
20 echo "Test 3: Simple validation test..."
21 echo "✅ PASS: This test always passes"
22
23 echo "==== ALL TESTS PASSED ====="
24 exit 0
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

